From Prepared Notes to a Stream: Porting a Physically Modelled Piano from C++ to Faust, Checked Against the C++ Model Itself

Stéphane Letz
GRAME-CNCM, Lyon
letz@grame.fr

October 2026

Abstract

`coopiano` is a physically modelled piano written in C++17: a nonlinear felt hammer, unison strings with two polarizations, longitudinal phantom partials, action noises, dampers and a sustain pedal, a sympathetic string register, a modal soundboard imaged over four bridge regions and a feedback-delay room. It prepares each note offline, with a contact solve oversampled 32 times followed by a bank of decaying phasors. We port it to Faust as a stream, with the Rust Faust compiler faust-rs, and check the port against the C++ model itself: a small program includes its headers unchanged and writes the tables, the renders and the intermediate signals the comparisons need. Each string mode becomes a complex one-pole resonator driven with a first-order hold; the hammer is solved implicitly at the audio rate, with the felt's hysteresis rate taken at the same sample, which a one-sample lag would raise by 41 % in the fortissimo treble; a phantom partial is the product of two mode states. Over the keyboard the hammer's peak force is within 1.1 % of the C++ solve (an 11-sample contact excepted), the bridge force within 0.3 dB, the sympathetic register identical to 10^{-12}, and the whole instrument within 0.1 dB of the C++ engine with the same stereo correlation. The soundboard is the C++ board's 732 modes with amplitudes fitted per bridge region: fitting the shared response and weighting it per region, the obvious shortcut, put the instrument 3.5 to 9.3 dB off. The port also measured the compiler: function applications re-evaluated at every use, a normal form cubic in the length of a sum, and per-block coefficients that cost more than the strings; the paper says how the model is written around each, and what faust-rs should change.

1 Introduction

Porting a synthesizer between languages is usually checked by listening. A physically modelled piano makes that hard: two renders that differ by the random draws of a strike, the phase of a beating unison or the realization of a filtered noise do not sound identical even when both models are right, and two that sound alike may still disagree by a few decibels in a register nobody played. We take the opposite stance. The C++ model, `coopiano` [1], is its own reference: a short program, `ref/dump.cpp`, includes its headers `piano.hpp` and `instrument.hpp` unchanged and writes everything the Faust model needs or should match: the fixed random tables, the per-key tuning draws, the post-contact excitation levels, the soundboard's modes and responses, single-note bridge forces, the force after the sympathetic register, and the stereo output of the full engine. The Faust model is then measured against these with `faustprobe`, the offline probe of faust-rs [6], which compiles a program, sets its controls, renders it and prints numbers.

The C++ model was not written for streaming. Each strike is *prepared*: a 32-times oversampled integration of the hammer against every mode of every unison string, after which the modes are converted into decaying phasors, the phantom partials are added as further phasors, and the action noises are written into an attack buffer. A Faust program has no "before the first sample": it computes every signal at every sample. The port is therefore a reformulation, and this paper is about the reformulations that kept the C++ model's numbers (Sections 3–4), the ones that did not and why (Section 5), and what the compiler made hard (Section 6).

2 The C++ model

A key is described by about twenty register-dependent parameters (speaking length, tension, inharmonicity, strike position, decay times, hammer mass and felt law) interpolated over the note number, scaled by the thirty voicing controls of `default.ini`. Keys below MIDI 31 have one string, below 41 two, the others three, detuned by per-key draws.

Strings. Each string is a sum of modes up to 17.5 kHz, at most 600. A mode's frequency follows the stiff-string law, pulled by the reactive part of a random modal bridge admittance; its loss has a late-stage rate plus a frequency-dependent term. The hammer cou-

ples to a mode through the strike-point mode shape, softened by the felt's width; the bridge reads it through a weight carrying an empirical register-dependent voicing.

Hammer. The felt law is $F = K\delta^p(1 + 0.12\,\dot\delta)$ for a compression $\delta > 0$, a power law with a Hunt–Crossley-like hysteresis [8, 10]. `run_contact` integrates the hammer and all modes by the symplectic Euler method at 32 times 44.1 kHz until the hammer separates [4].

Free vibration. At separation each mode's state is split into a vertical polarization, which carries most of the energy and decays fast (its rate follows the bridge admittance's real part, so modes near a board resonance drain sooner), and a horizontal one, slightly detuned and decaying at the late rate: the two-stage decay of real pianos [11]. Both become phasors $z \mapsto z\,e^{(-\sigma+i\omega)/f_s}$; the damper blends each phasor's step toward a seated-damper rate over 40 ms.

Phantom partials. Longitudinal coupling is quadratic in the string slope, so pairs of modes drive the longitudinal string at their sum frequency [2, 5]. `add_phantoms` forms, for neighbouring partials $i \le j \le i + 3$ below 4.2 kHz and keys below G#3, the product of the two phasors, scaled, resonantly boosted near the longitudinal modes and bounded.

Body. The engine sums the voices, adds the response of a sympathetic bank (88 keys, partials below 7 kHz, open when a key is held, the pedal is down or the key has no damper; a key's own force is withheld from its own resonators), splits the force over four bridge regions and convolves it with the soundboard: 732 modes from 48 Hz to 16.5 kHz equalized by a minimum-phase filter that also removes three quarters of the narrow modal ripple, imaged per region for the first 0.2 s and shared afterwards. A DC filter, an eight-line feedback delay network [9], an output gain and a soft limiter follow.

3 The voice as a stream

3.1 Modes as complex one-poles

A mode obeys $\ddot q + 2\sigma\dot q + (\omega^2 + \sigma^2)q = gF(t)$. With the C++ convention $z = q - i(\dot q + \sigma q)/\omega$ (`polarization_state`), $\mathrm{Re}\,z$ is the displacement and

$$\dot z = p\,z - i\frac{g}{\omega}F, \qquad p = -\sigma + i\omega. \tag{1}$$

Over one sample T, with F linear between F_{n-1} and F_n (a first-order hold),

$$z_n = \lambda z_{n-1} + G_0F_{n-1} + G_1F_n, \quad \lambda = e^{pT}, \tag{2}$$

with $G_1 = b\,T\varphi_2(pT)$, $G_0 = b\,T(\varphi_1 - \varphi_2)(pT)$, $b = -ig/\omega$, and the functions $\varphi_1(x) = (e^x - 1)/x$, $\varphi_2(x) = (e^x - 1 - x)/x^2$ of exponential integrators [7], evaluated by their Taylor series near zero. The recursion is exact for a piecewise-linear force, so a mode's free decay is exactly the C++ phasor's.

The hammer needs the strike point's displacement *before* the current force acts. Keeping z_n as state would recompute it every sample; the voice keeps instead the free prediction $w_n = \lambda z_{n-1} + G_0F_{n-1}$, which obeys

$$w_n = \lambda w_{n-1} + (\lambda G_1 + G_0)F_{n-1}, \tag{3}$$

so the strike point's free displacement, the bridge force and the phantoms all read w directly; they differ from z_n by G_1F_n, a fraction of one sample's motion, and only during contact. This change of state halved the voice's cost.

Polarizations. The C++ splits the state at separation; splitting the *drive* in the same proportions, into a vertical and a horizontal resonator, gives the same free vibration, since the modes are linear; during the few milliseconds of contact the two resonators decay at different rates, a difference the contact comparison of Section 5 does not see.

Slots. Faust needs the number of resonators at compile time. A voice has $N_S = 96$ slots shared by its strings, string by string: with n_s strings, slot k is partial $k \bmod (N_S/n_s) + 1$ of string $\lfloor k/(N_S/n_s)\rfloor$. A one-string bass key gets 96 partials, a trichord 32 per string. Every frequency, loss, decay share and bridge weight of `build_string` and `collect_phasors` is a function of the note and the voicing controls.

3.2 The hammer, implicit at the audio rate

At 44.1 kHz the treble contact lasts a few dozen samples and the felt is stiff: for C8 at 20 N we estimate the loop gain of an explicit scheme (the felt's tangent stiffness times the string's response over one sample) at about 6, far beyond what a force computed from the previous sample's compression can follow. The voice solves the delay-free loop [3] instead. The strike point's displacement is $y_s = y_s^{\mathrm{free}} + R_sF_n$ and the hammer's $y_h = y_h^{\mathrm{free}} - R_hF_n$, where $R_s = \sum_k \bar a_k \mathrm{Re}\,G_{1,k}$ sums over the modes (readout $\bar a_k$ averaged over the unison, which receives equal shares of the force) and $R_h = T^2/6m_h$ for the same hold. The compression is $\delta = a - cF_n$ with $a = y_h^{\mathrm{free}} - y_s^{\mathrm{free}}$ and $c = R_h + R_s$. Its rate is linear in F_n too, $\dot\delta = d_0 - d_1F_n$, from the free velocities and the direct velocity gains of the same modes. The felt law then gives, with $A = n_sK(1 + 0.12\,d_0)$ and $E = 0.12\,d_1n_sK$,

$$\delta + c\,\frac{A\,\delta^p}{1 + E\,\delta^p} = a, \qquad F_n = \frac{A\,\delta^p}{1 + E\,\delta^p}, \tag{4}$$

a scalar equation with a single root in $(0, a]$. Newton's method starts from the smaller of the two one-term solutions, $\min(a, (a/cA)^{1/p})$, which lies above the root, and five steps, kept in $[\delta/2, a]$, are enough.

Taking the rate $\dot{\delta}$ one sample late is tempting, since everything else is solved. It is not innocent. A numpy replica of both schemes on the C++ modes (`scripts/contact_sim.py`), which reproduces the peak forces `faustprobe` measures on the Faust voice to 0.05 %, shows the lagged rate raising the fortissimo peak force by 41 % at C6 and 42 % at C7 while leaving the softer strikes within 1 % (Fig. 1): the hysteresis term is largest at the start of a fast contact, where a one-sample lag is a large fraction of the compression time.

3.3 Phantom partials as products

A phantom phasor is the product of two parent phasors, so its frequency and decay are the sums of theirs. In a stream, the product of the two parents' current states *is* that phasor, at no cost in state: released strings release their phantoms, and the damper's doubled rate on a phantom is the sum of its parents' rates. The voice forms the pairs of the same string and polarization at distances $d \le 2$ (the C++ allows 3; the next term is 0.55^3, 15 dB down), 288 pairs for 96 slots, each with a coefficient carrying the bridge voicing at the sum frequency and the longitudinal resonances. One identity matters for the compiler: the real part of $a\,b$ is written $(a_r - a_i)(b_r + b_i) + a_i b_r - a_r b_i$, because for $d = 0$ the textbook form contains a_r^2, which faust-rs normalizes to `pow(x, 2)`, a library call in its JIT: 391 of them per sample in the first version.

3.4 What comes from tables

Some quantities are random draws or the result of a solve the stream cannot redo. `ref/dump.cpp` writes them and `scripts/gen_tables.py` turns them into `coopiano_tables.lib`: the bridge admittance at 1 Hz steps (a random modal sum), the per-key detuning draws, and `tone_peak_amp`, the excitation level after contact that scales every action noise, for 88 keys and 25 velocities (mean over four seeds). The action noises (keybed thump, structural knock, felt scrape, treble knock, damper felt) follow `prep_note_into`'s filters and envelopes on Faust noise.

One draw is per strike: each string's bridge readout is scaled by $1 + 0.13u$, u uniform. The Faust voice samples it at the strike. Left to that, the first comparison showed every multi-string note 0.3 to 0.5 dB louder than the C++: the C++ renders use seed 1, whose first draws are -0.73, -0.73, -0.10. A test-only entry point, `voice_drawn`, takes the draws as arguments; with the seed's draws the offset vanished. A systematic offset can be a sample of one random draw.

4 The body

Inputs and chain. The body is the program's `effect`: the polyphonic architecture sums the voices' outputs channel by channel and runs it once on the sum, as the engine sums its voices before its body; this is exact because everything before the limiter is linear. A voice has five outputs (Fig. 2). Four carry its bridge force F split over the bridge regions: the key sits at $(m - 21)/87$ along the bridge and its force is crossfaded linearly between the two regions around that point (centres at 1/8, 3/8, 5/8, 7/8), the engine's `key_region_`; A0 feeds region 0 only, C8 region 3 only. The gains sum to one, so the four channels add up to the total force. The fifth is the correction of the played key, described below. The body adds the sympathetic register's output to every region, drives the soundboard with the four region forces, and passes each output channel through the DC filter, the room, the output gain and the limiter:

```
_body_with(board, x0, x1, x2, x3, c)
  = (x0 + w, x1 + w, x2 + w, x3 + w) : board
  : dc_block, dc_block : room
  : *(_out_gain), *(_out_gain) : limit, limit
with { w = (_r_gain * res_bank(x0 + x1 + x2 + x3)
           + c) / REGIONS; };
```

Sympathetic register. The bank transcribes the C++ class `SympatheticResonance`: 1002 resonators, up to 16 partials per key, excited below 7 kHz only. Partial p of a key of fundamental f_0 and inharmonicity B is a damped rotation at $f = p f_0 \sqrt{1 + B p^2}$. Its radius is interpolated by the dampers' openness o between the open radius r (the key's T_{60}, at most `maximum_decay_seconds`, 12 s) and the closed one (T_{60} of `damper_decay_seconds`, 0.18 s). Its drive $2(1 - r)/\big(p(1 + (f/3500)^2)\big)$ is multiplied by o, so a seated damper lets the string decay without feeding it. The openness follows $1 -$ `damper_contact`$(pedal)$ at a bounded rate (`transition_seconds`, 40 ms); keys from F#6 up have no damper and stay open, so with the pedal up only they ring. The 69 keys with dampers each run in an `ondemand` domain, awake while the dampers are lifted and for six closed T_{60} after they seat: a sleeping key costs nothing, outputs zero, and recomputes its parameters when it wakes. The output, scaled by `gain` (0.18), is spread evenly over the four regions, as `render_force` spreads whatever the regions do not account for. A Faust effect sees only the summed force, not which keys are held, so two of the C++ behaviours need care. The withholding of a key's own force is linear: $\sum_k R_k(x - w\,x_k) = \sum_k R_k(x) - w \sum_k R_k(x_k)$, and the second term is computed by each voice from its own force, with the openness its key has in the shared bank, and handed to the body on a fifth channel. Driven by the C++ force of a C4 strike, the Faust bank and correction equal the C++ bank to 10^{-12} (-248 dB). The opening of a key held silently is not modelled.

Soundboard. The C++ board is a modal sum followed by an equalizer. The obvious port evaluates the equalizer at each mode's frequency and keeps the modes. It fails, with an error 1.3 times the response itself: the equalizer removes three quarters of the narrow ripple, so it is not smooth at the scale of a mode, and it is precisely at the modal peaks that it cuts. The

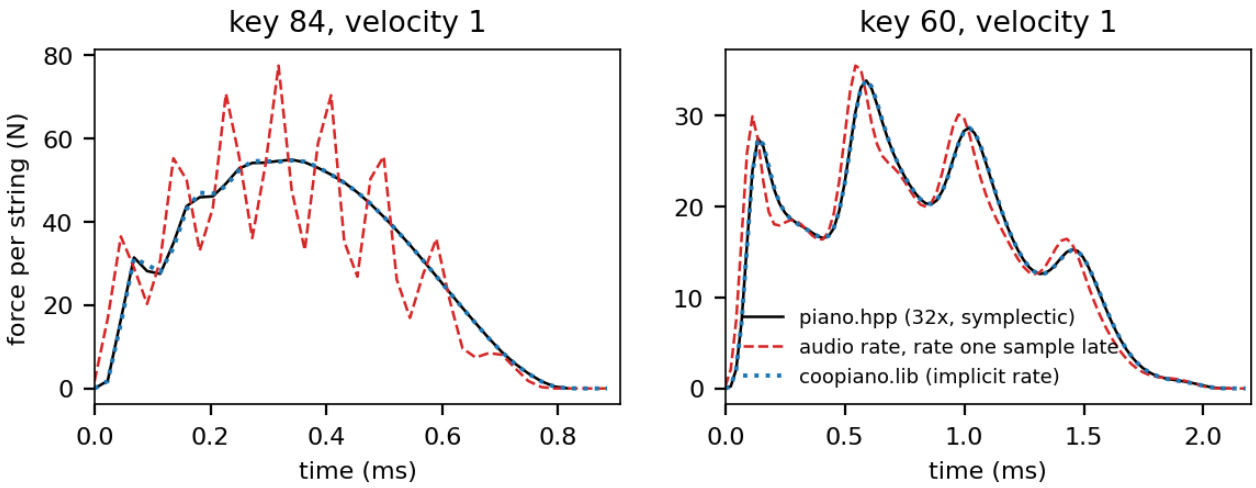


Figure 1: Hammer force per string at velocity 1. Black: `run_contact` (symplectic Euler at 32 times 44.1 kHz). Red: the audio-rate implicit scheme with the felt's hysteresis rate taken one sample late. Blue: `coopiano.lib`, rate taken at the same sample.

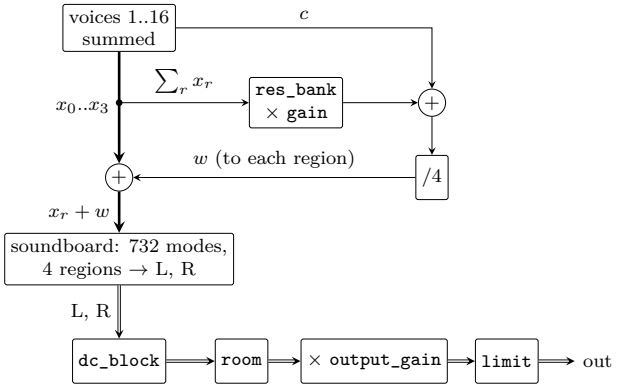


Figure 2: The body. Thick lines carry the four region forces, double lines the two output channels. The voices' fifth channel c withholds each key's own share from the sympathetic register.

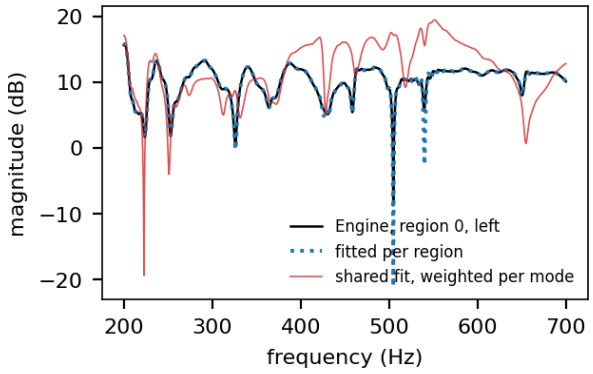


Figure 3: Magnitude of region 0's left response (Engine), its per-region fit, and the shared fit weighted per mode as `make_soundboard_regions` weights the modes.

port keeps the 732 poles and fits their complex amplitudes by least squares to the C++ response. The Gram matrix of damped exponentials has a closed form,

$$\sum_{n=0}^{N-1} \lambda_k^n \lambda_l^n = \frac{1-(\lambda_k\lambda_l)^N}{1-\lambda_k\lambda_l}, \qquad (5)$$

and its conjugate counterpart, so no 67914×1464 matrix is formed; one Cholesky factorization serves every target. The shared response is fitted to -25.5 dB.

The regions are where the port went wrong first. `make_soundboard_regions` weights each mode per region (a random smooth mode shape along the bridge and a constant-power pan) before the equalizer. Weighting the *fitted* amplitudes in the same way seemed equivalent. It is not: neighbouring modes overlap, the least-squares amplitudes are not unique, and the fit's cancellations between neighbours do not survive a weighting. Fed the exact region forces of a C4 strike, that board was 3.5 to 9.3 dB too loud in the engine's output, depending on the channel and the time window; on a region's response the magnitude error is visible across the spectrum (Fig. 3). Each region's response (head and shared tail, as the engine hears an impulse in that region) is now fitted on its own, for both channels: eight targets, one Gram matrix, -22.8 to -25.7 dB. A mode is driven by the complex combination $\sum_r A_{r,k} x_r$ of the region forces, which costs as much as two real drives: each mode has one complex one-pole per channel c, $s_c[n] = \lambda_k s_c[n-1] + \sum_r A^c_{r,k}\, x_r[n]$, read through $\operatorname{Im} s_c$, 1464 resonators in all, whose parameters are read from the tables once, at the first sample, in twelve `ondemand` domains of 61 modes. The stereo image is the difference between a region's left and right amplitudes. The imaging is fixed at the C++ defaults (spread 0.7, position depth 0.5). A light body without imaging sums the regions first and reads a single resonator u per mode through two amplitudes, $\operatorname{Im}(a_L u)$ and $\operatorname{Im}(a_R u)$, for about a third of the board's cost.

Room and output. A DC filter, a one-pole high-pass at 20 Hz, comes first. `StereoReverb` is then transcribed line by line: the wet path is high-passed at 150 Hz and predelayed by 12 ms; six early taps between 7 and 31 ms cross the channels; two diffusion all-passes per side feed eight lines of 1493 to 2423 samples with an unnormalised Walsh–Hadamard feedback, each damped by a one-pole low-pass at 5.5 kHz and scaled for a 1.1 s decay. The output is the dry signal plus $0.3\,(0.45\,e + 0.70\,\ell/\sqrt{8})$, with e the early and ℓ the late part. The output gain (0.0011, from newtons to audio level) and the soft limiter follow; the limiter, unchanged below 0.8 with a tanh knee toward 1 above, is the only nonlinear stage, which is what allows the voices to be summed before the body.

5 Results

Table 1 gathers the comparisons, all produced by `make results`; the figures come from the same renders.

The contact, the notes (Fig. 4) and the engine (Fig. 5) agree where the models are the same model. The differences that remain have causes:

- *Partials per string.* A trichord has 32 partials per string; C3 loses the C++ content above 4.5 kHz, at most −44 dB of the energy at the attack, which shows as the 9.6 dB rms spectral difference over the bands above 4 kHz. $N_S = 144$ gives 48 partials per string (−67 dB) for 1.5 times the voice's cost.
- *The shortest contacts.* At C8 fortissimo the contact lasts 11 samples and the audio-rate scheme loses 14 % of peak force and 3 % of impulse.
- *Phantoms.* The −80 dB pair threshold is not applied. The per-pair bound of `bound_phantom` is applied once per strike, from the parents' amplitudes at the end of the contact, computed in an `ondemand` domain clocked by the contact and held after it; without it, A0 and C1 fortissimo were 2 to 3 dB louder at the bridge.
- *Release tails.* After a release the C++ board rings on, mostly between 300 Hz and 1 kHz: its minimum-phase equalizer, designed on a 0.34 Hz grid, lifts three quarters of the board's narrow nulls above 450 Hz by up to 20 dB, and these narrow peaks ring for about 2 s until the response is cut at 1.4 s. Its response is at −48, −56 and −72 dB over 0.3–0.5, 0.5–0.75 and 1–1.4 s, the modal fit's at −65, −92 and −158 dB; the modes (T60 at most 0.45 s) cannot carry that ring, and 64 resonators fitted to it gain 1 to 2 dB. After a staccato the Faust output is 15 to 25 dB lower from 0.35 s after the release, masked by the room reverb for about a second.
- *Noises.* Action noises are other realizations of the same filtered noise, and their level comes from the default voicing's contact solve.

6 What the compiler made hard

The first complete voice did not compile in useful time: 435 s, then a failure of the Cranelift JIT on a too large `compute` function. Each obstacle was measured and written around; together they are a report on faust-rs.

Evaluation does not memoize application. The voice was first written as functions of the note, the slot and the polarization, called wherever a quantity was needed, many of them through lambdas (`force_gains(...) : \(g0r, ...).(...)`). The generated code was right, each resonator once, thanks to hash-consing. But the evaluator caches a call by expression and environment *identity*, and every application of a closure opens a new environment: a call inside a lambda body is re-evaluated at each application, and a chain of depth D costs 2^D. The voice took about 0.9 s of evaluation per slot; even rewritten as dataflow (each slot a box evaluated once, publishing its signals on a bus that `route` distributes to the sums and the pairs) it still peaks at 17 GB during evaluation, enough to make three concurrent compilations five times slower. The C++ compiler solved the same problem by memoizing `applyList` on the closure and its evaluated arguments; faust-rs has not ported it yet.

Long sums. The normalization phase, `fir-prepare-normalize`, grows as the cube of a sum's length: at each partial sum of the left-nested chain `:>` builds, the search for a common factor tests every pair of terms, and faust-rs runs the simplification twice. The soundboard's two 732-term sums cost 16 s, and 1.6 s once summed in groups of 32 behind an opaque `max(x, -1e300)`.

Per-block coefficients. Faust recomputes every control-rate expression at each block. For this voice that is 96 slots and 288 phantom coefficients: 11 500 transcendental calls per block, about 180 per sample at 64-sample blocks, more than the strings. faust-rs's `ondemand` runs a sub-program only on the samples where its clock fires. Clocked by "the note or the voicing changed", it would recompute the coefficients only then, except that the compiler hoists a computation depending only on controls out of the domain, back to block rate. Entering the note as a held audio-rate signal keeps it inside:

```
on_change(chg, x, f) = (chg, ba.sAndH(chg, x)) : ondemand(f);
params = voice_params_on(chg, m);  // chg: note or voicing
      changed
```

The voice ran 1.8 to 2.7 times faster (bass and middle keys). The same mechanism lets the phantoms run only for keys below G#3, and the sympathetic resonators of keys with dampers only while the dampers are lifted or for a second after; a sleeping key recomputes its parameters when it wakes, so a voicing change made while it sleeps is not lost.

Smaller traps. `rad` and `fad` are reserved words (the differentiation primitives), which the C++ compiler accepts as identifiers, and the parse error does not say so; `-max(a, b)` does not parse, as in C++, with an obscure message; `A : f, B : g` reads as `A : (f,`

Comparison	Result
Hammer contact, 8 keys × 4 velocities, against `prep_note`	peak force within 1.1 %; contact time within 1.3 % except at C8 (5 to 6 %, under one sample); C8 at velocity 1: −14 % peak force, an 11-sample contact
Bridge force, 12 notes from A0 to C8 (velocities 0.2 to 1), against `render_bridge_force`	level within 0.1 dB in every window after 50 ms, a release included (windows floored at −100 dB of the peak), 0.3 dB over the first 50 ms; third-octave spectra within 0.6 dB rms, except the attack of C3 forte (9.6 dB rms over bands above 4 kHz, see text)
Bridge force, all 88 keys at velocity 0.5, held 2 s, against `render_bridge_force` (`make keyboard`)	level within 0.4 dB over the first 50 ms and 0.1 dB from 50 ms to 1 s; within 0.7 dB from 1 to 2 s (the top octave's tails, 50 to 65 dB down); third-octave spectra within 0.9 dB rms early and 1 dB rms late, except D7 to E7 late (up to 5.7 dB, a weak partial of a tail 58 dB down)
Sympathetic register, driven by the C++ force	equal to 10^{-12} (−248 dB)
Soundboard fit	shared response −25.5 dB; regions −22.8 to −25.7 dB; magnitude error 0.1 dB median, under 0.7 dB for 90 % of frequencies (50 Hz–10 kHz)
Whole instrument against `Engine`, C4, velocity 0.5, held 1 s, dry and with reverb	level within 0.1 dB per channel while the note sounds; left/right correlation 0.355 and 0.311 for both models; reverb tail within 0.1 dB

Table 1: The Faust model against the C++ model (`results/*.txt`).

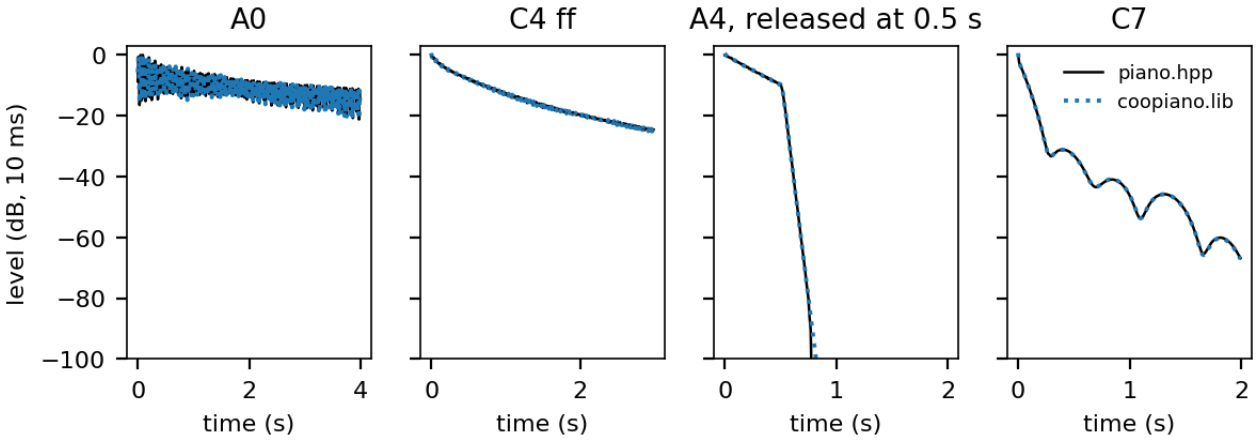


Figure 4: Bridge force level (10 ms windows) of four notes, `render_bridge_force` and the Faust voice with the seed's draws.

`B) : g`, and the resulting arity error was reported on a shared `(_, _)` of an unrelated definition; `x*x` becomes `pow(x, 2)`, emitted as a library call where the C++ compiler emits a multiplication; `faustprobe`'s polyphonic mode panics when `effect` changes the channel count, so the instrument is checked by rendering the voice to a five-channel file that then drives the body; and `-vec` does not finish on the voice, its vector planning being quartic in the number of loops.

Since then. Reported with minimal reproducers, most of these obstacles were removed from faust-rs on the day this article was finished: application is memoized as in the C++ compiler; the long sums, the merging of isomorphic recursions, the recursive type fixpoint and the clock fixpoint no longer cost the number of recursion groups times the graph; `x*x` is a product on every backend; `faustprobe` runs the instrument with its body; and the diagnostics name the reserved words, explain the unary minus and locate the faulty composition. The voice now compiles in 7 s and 2.8 GB instead of 41 s and 17 GB, the body in 5 s instead of 45 s, and the two as one program in 13 s instead of 363 s, with the same generated code and the same comparisons (Table 1). The voice's dataflow style, the grouped sums (still 0.5 s faster on the body) and the square-free phantom formula are kept. Two items remain: `-vec` on the voice, whose vector planning does not end on its 31 825 loops, and the hoisting out of `ondemand`, which `on_change` still works around.

Cost. Compiled by faust-rs to C++ and by clang at `-O3`, a sounding voice costs 3.1 % of an Apple M5 core. The imaged body costs 36 % of a core on the demo score without pedal and 40 % with it (the dampers' resonators then wake up); the light body 10 % and 15 % (`results/cost.txt`, best of three runs). The C++ engine convolves its board by partitioned FFTs; a modal board is the price of a model whose every sample is computed in Faust.

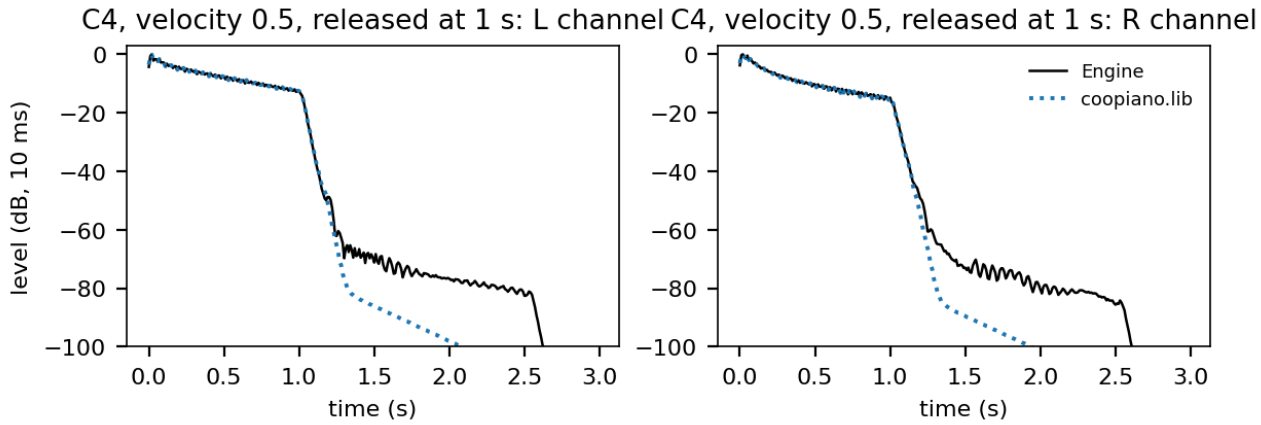


Figure 5: The whole instrument: one strike through the sympathetic register, the imaged soundboard and the output stage, against coopiano's `Engine`.

7 Conclusion

A model that prepares its notes can be ported to a stream without losing its numbers, if the port is checked against the model and not against an ear: the hammer, the strings and their two polarizations, the sympathetic register and the body agree with the C++ to a tenth of a decibel. The checks found the two errors that listening would have hidden: the hysteresis lag that inflates the fortissimo treble by 40 %, and the shortcut that moved the instrument's level by 5 to 9 dB through the soundboard's regions. They also found a seed. On the compiler side, the port is a list of cases for faust-rs: an application memo in the evaluator, a normal form for long sums, and coefficients recomputed when their inputs change rather than at every block, which `ondemand` can express today with a held input. The first two have since been fixed.

References

[1] coopiano: a physically modelled piano synthesizer in C++17, 2026. MIT license, revision `bba9e06`, 2026.

[2] Balázs Bank and László Sujbert. Generation of longitudinal vibrations in piano strings: From physics to sound synthesis. *Journal of the Acoustical Society of America*, 117(4):2268–2278, 2005.

[3] Gianpaolo Borin, Giovanni De Poli, and Davide Rocchesso. Elimination of delay-free loops in discrete-time models of nonlinear acoustic systems. *IEEE Transactions on Speech and Audio Processing*, 8(5):597–605, 2000.

[4] Antoine Chaigne and Anders Askenfelt. Numerical simulations of piano strings. I. A physical model for a struck string using finite difference methods. *Journal of the Acoustical Society of America*, 95(2):1112–1118, 1994.

[5] Harold A. Conklin. Generation of partials due to nonlinear mixing in a stringed instrument. *Journal of the Acoustical Society of America*, 105(1): 536–545, 1999.

[6] GRAME-CNCM. faust-rs: the Faust compiler in Rust, 2026. Release build of 15 October 2026.

[7] Marlis Hochbruck and Alexander Ostermann. Exponential integrators. *Acta Numerica*, 19:209–286, 2010.

[8] Kenneth H. Hunt and F. R. Erskine Crossley. Coefficient of restitution interpreted as damping in vibroimpact. *Journal of Applied Mechanics*, 42 (2):440–445, 1975.

[9] Jean-Marc Jot and Antoine Chaigne. Digital delay networks for designing artificial reverberators. In *Proceedings of the 90th Convention of the Audio Engineering Society*, 1991. Preprint 3030.

[10] Anatoli Stulov. Hysteretic model of the grand piano hammer felt. *Journal of the Acoustical Society of America*, 97(4):2577–2585, 1995.

[11] Gabriel Weinreich. Coupled piano strings. *Journal of the Acoustical Society of America*, 62(6): 1474–1484, 1977.